

Secure EHR Management System

● System Workflow

Our system is a web application. We have three types of users in our system; Admin, Doctor, and Patient.

Admin:

- Security Professionals and Auditors can be considered Admins
- Admins can access the audit data for any patient that they want.

Patients:

- Patients are the people whose EHR data accesses are being logged.
- Every Patient can check their EHR data along with the audit logs of who performed what action on their EHR data.

Doctor:

- Doctors treat the patients and hence, can modify the EHR data.
- They can read, add, and delete entries into the EHR record of a patient.

Workflow for Patient:

- The user will first go to the web application and will land on the login page.
- They will then enter their credentials and hit submit.
- The credentials will be sent to the authentication server to be verified.
- If verification is successful, the user will be redirected to the patient dashboard that contains the Query button.
- If verification fails, the user is presented with "Invalid Credentials"
- On clicking the Query button, the patient gets information on who performed what action on their EHR data.
- The current access event is added too. (Patient accessed their records).

Workflow for Admin:

- The user will first go to the web application and will land on the login page.
- They will then enter their credentials and hit submit.
- The credentials will be sent to the authentication server to be verified.
- If verification is successful, the user now verified as admin will be presented with an Admin dashboard with a Query button and a textbox to add the user's name for whose EHR logs need to be checked.
- If verification fails, the user is presented with "Invalid Credentials"
- The Admin can enter any of the patient's name, hit the query button and they would be shown EHR logs for that patient.

Workflow for Doctor:

- The user will first go to the web application and will land on the login page.
- They will then enter their credentials and hit submit.
- The credentials will be sent to the authentication server to be verified.
- If verification is successful, the doctor is presented with the doctor dashboard which has three sections.
- If verification fails, the user is presented with “Invalid Credentials”
- The first section is to enter a patient’s name and they will be presented with that patient’s EHR data. Any access to the data is added to the log file
- The second section is to enter a patient’s name and an update to their EHR data. The doctor can add any changes to their EHR data. For example, the name “John Doe” and the comment will be “has COVID”. This will now be added to John Doe’s EHR record and an entry will be made into the audit file.
- The third section is to enter a patient’s name and delete a particular entry in their EHR data. For example, John Doe had COVID, but he completed his quarantine period and is now healthy. The name will be “John Doe” and the comment to be deleted will be “has COVID”. An entry of this deletion will be added to the audit file.

- **System Architecture**

Web application:

- The user interacts with the system using the UI made available through our web application created using flask.
- The user first arrives at the login page to prove their identity.
- The dashboards and the results of queries are displayed on this web application.

Authentication Server:

- The username and hash of the password that the user enters into the login page are sent to our authentication server code.
- The username and hash of passwords are stored as a key-value pair in our authentication server.
- The username is searched for in the dictionary and the corresponding hash of the password is found.
- If the hash of the password entered matches the hashed password in the authentication server, the user’s identity is verified.
- Based on which list of the three types of users (Doctor, Admin, or Patient) the user logging in belongs to, the authentication server communicates to the web application the type of dashboard to redirect the user to.
- If they do not match, the user is redirected to a page that says invalid credentials.

Query Server:

- After successful authentication from the authentication server, the web application displays the dashboard to the user based on the type of user.
- For patients, when they hit the Query, the query server will read from a file containing patient EHR information. The query server will also request the audit server to retrieve the audit logs for that user. Both, the EHR data and decrypted audit logs for that user are returned to the web application on the dashboard for the patient to view.
- For doctors, when they hit Query for a particular patient, the query server will read from a file containing patient EHR information and return that information to the dashboard. The record of the access request of EHR data is sent to the audit server.
- When the doctor adds information to a patient's EHR, the added information is written to the patient's EHR file, and the record of this addition is sent to the audit server.
- When the doctor deletes information from a patient's EHR, the information to be deleted is searched for in the patient's EHR file and then deleted. The record of this deletion being made is sent to the audit server.
- For admins, when they hit Query for a particular patient, a request is made to the audit server for the audit logs to the EHR of that patient. The dashboard is returned with audit logs for the specified user for the admin to view.
- All communication from the query server to the audit server is encrypted.

Audit Server:

- The audit server is responsible for writing to and retrieving the audit log files.
- The requests from the query server when a patient requests for their EHR access logs are sent to the audit server. The audit server decrypts the master audit file and adds the current access request to it. Then it searches for the logs about that specific patient and then extracts that data. This extracted data is sent to the dashboard of the web application.
- When a doctor requests access to the EHR data of a patient from the query server, the query server sends this access request to the audit server. The audit server will add this access record to the master audit file.
- When a doctor adds or deletes information from the EHR data of a patient, the query server sends this request to the audit server. The audit server will add this modification record to the master audit file.
- When an admin requests audit logs for a specific user, the master audit file is decrypted, then the logs about that user are searched for and extracted. The extracted logs are returned to the dashboard of the web application.
- All communication from the audit server to the query server is encrypted.
- Any new data added to the master audit file leads to a new calculation of the master hash.
- This master hash is used for integrity checks any time a user queries something or updates the EHR data.

- **Cryptographic components**

Authentication:

- The authentication of our users is done by using a simple hash-based authentication.
- When a user is introduced into the system, the hash of their entered password is calculated and stored.
- Every time a user wants access to the system, they need to enter their username and password.
- The SHA256 hash of their password at the time of login is calculated and compared against the stored value.
- If it matches, the user has proven their identity and is successfully authenticated.
- If it doesn't, authentication fails and access is denied.

Encryption at rest:

- The audit file is encrypted using AES256 and stored.
- This ensures that if anyone opens the audit file, they won't be able to decipher it.
- Whenever the audit server needs to extract the audit files of a user, the symmetric key is used, the file is decrypted and the information is extracted.
- Whenever the audit server needs to write to the audit file, the symmetric key is used, the file is decrypted, the new data is added to the file, and then encrypted again.

Encryption during communication:

- Updating the audit file:
 - When a user wants to access any EHR data, their access record along with action and timestamp is sent from the query server to the audit server.
 - This record is encrypted by the query server using a random AES256 key.
 - The encrypted record along with the AES key is encrypted using the public key of the audit server (using RSA).
 - The audit server receives this encrypted communication and uses its private key to decrypt the AES key.
 - The AES key is then used to decrypt the access record.
 - The data at rest (master audit file) is decrypted, and this communication from the query server is written to the master audit file by the audit server.
 - The master audit file is then encrypted again.
- Extracting logs:
 - When a patient wants to view their own logs or an admin wants to view the logs of a particular patient, the request from the query server is sent to the audit server.
 - The audit server decrypts the master audit file at rest using the AES256 key.

- The audit server then searches the audit file for all the logs related to that specific user and extracts it.
- The master audit file at rest is then encrypted using the AES256 key.
- The extracted log files are encrypted using a random AES256 key.
- The AES key is encrypted by the audit server using the public key of the query server (using RSA).
- That communication is sent to the query server.
- The query server decrypts the AES key using its private key (using RSA).
- The query server then decrypts the requested logs using the AES256 key and presents them on the dashboard to the user.

Integrity checks:

- For any query being made or updates to EHR data, the integrity of the master audit file is checked.
- First, for the current state, the SHA256 of the file is calculated and stored. We are calling this the master hash.
- Whenever any query is made, the SHA256 hash over the decrypted log file is calculated.
- This hash is compared with the master hash.
- If it doesn't match, the user querying is notified.
- If it does match, the new access request if any is added to the file, and a new hash is calculated over the master audit file.
- This becomes the new master hash.

● **Additional Implementation**

Homographic Encryption:

- Homographic Encryption lets you perform operations on encrypted data.
- The data can be added and multiplied without actually having to decrypt the data first, which reduces the risk of the data being leaked.
- This is particularly useful where privacy and confidentiality is a top priority like while dealing with EHR data.

Use Cases in our system:

- Suppose we introduce vendor data analyst team in our system.
- The job of these analysts is to analyse the log data for entries of people who had cancer in the last 3 months.
- Since the analyst team are vendors from another organization hired on contract basis, we do not want to give them too much confidential information.
- We do not want them to know who the patients with cancer are, but simply the people who have recently been tested positive for cancer.
- Using homographic encryption, they can find all the entries of “positive for cancer” added by the doctors from the encrypted audit file.
- They can add these entries and get the total number without any personal identification information of the people who tested positive for cancer.

- Suppose we introduce a security analyst into the system with similar privileges as the ‘admin’ account.
- However, we do not want this analyst to be able to view sensitive EHR data with PII since he is a new intern.
- His job is to monitor the amount of logs being generated and ensure that if too many logs are being generated, he should report it to his manager for further investigation.
- This can be done by using homographic encryption and adding the logs. The intern would not get any information about the logs, but only the total amount of logs being generated in the last hour.

Incorporating Homographic Encryption:

- Choose a library that supports homographic encryption (Microsoft SEAL)
- Introduce data structures into the system to hold homographically encrypted audit records.
- Modify the query and audit server to handle homographic encryption operations. This includes changing how data is read and processed.
- Only authorized users should be able to perform encryption operations like the data analysts or security analysts.

Hence, addition of homographic encryption in our system will be beneficial as it would allow data to be analyzed (increased functionality) while maintaining the confidentiality of the data.

Post-Quantum Cryptography:

- Post-Quantum Cryptography (PQC) consists of many cryptography schemes that are designed to be safe against attacks using quantum computers.
- The NIST has been pushing for their standardization.

- Lattice based encryption currently is the most promising in PQC due to its efficiency and security.
- In specific, we will be talking about Kyber.
- Kyber is a lattice based encryption and primarily used as a Key encapsulation mechanism (KEM)

Use Cases in our system:

- Kyber can be used to establish a session key between patients, doctors or admins and the servers.
- The session key can be used to securely transmit data during a session.
- Kyber can be used to encrypt data at rest.
- The audit records that are currently being encrypted by AES, can be replaced with Kyber's KEM ensuring the audit data is safe from any quantum based attacks.
- In case the EHR data from an organization using our system needs to be transferred to another organization (In the case that a patient takes a second opinion), the transmission of the EHR data can be done using Kyber.

Incorporating Kyber:

- To incorporate Kyber, we first need to access the current infrastructure and identify places we are using RSA. (Communications between Audit Server and Query Server)
- Deciding whether we are using Kyber directly by completely removing RSA or using a hybrid mode.
- Include libraries that support Kyber. Libops (Open Quantum Safe) is a library that provides implementation of Kyber.
- Use Kyber's key encapsulation to generate session keys when a user logs in (Patient, Doctor or Admin)
- Use these session keys derived from Kyber's key encapsulation to encrypt data for transmission or storage.

Hence, implementing Post Quantum Cryptography will make our systems secure for the future and not only in the current scenario. With quantum computing on the rise it is essential to be prepared for such attacks

Zero Knowledge Proofs (ZPFs)

- In Zero Knowledge Proofs, a prover can prove to a verifier that a given statement is true without actually revealing anything about the statement besides it being true.
- This can be particularly useful when there are multiple users in the system.

Use cases in our system:

- Our system currently uses a hash based authentication mechanism.
- This can be risky if an attacker is able to capture the hash being sent to the authentication server from the web application.
- We can replace this authentication mechanism with a zero knowledge proof based authentication.
- The user logging in can prove their password without actually sending the hash of the password to the authentication server.
- This greatly reduces the risk of password theft during transmission and ensures confidentiality of the passwords even over the hashed form.

Incorporating ZPF:

- Select a ZPF scheme for authentication, for example, Schnorr's Protocol.
- Establish the cryptographic parameters and secrets. Generate the secret key and public key.
- Public key is stored on the authentication server.
- When the user logs in, they prove that they know the secret key corresponding to their public key that was stored on the authentication server. They do this without revealing the secret key.
- The server will check this proof and validate it using the public key of the user logging in. If the proof is valid, the server will grant access without actually seeing the secret key.
- The authentication server needs to be modified to incorporate a ZPF authentication.
- The login interface needs to be changed to process ZKP challenges and verification.

Hence, using ZPFs in our system for authentication will greatly reduce the risk of passwords being compromised. It will also reduce the accounts that get compromised hence reducing the risk to the overall system.

- **Meeting the requirements**

Privacy:

- No user can query for any data without being authenticated first.

- The audit file at rest is encrypted using symmetric key encryption (AES 256). Hence, if anyone gets access to the file, they would not be able to read the data in it.
- The communication from the audit server to the query server while returning audit data for a specific patient is encrypted using AES 256. And the AES key is encrypted using the public key of the query server. Hence, anyone trying to sniff the conversation would not be able to determine the data that is going through.
- The communication from the query server to the audit server when someone accesses, adds or deletes to the EHR data is encrypted using AES 256. The AES key is encrypted using public key of the audit server. Hence, anyone trying to sniff the conversation would not be able to determine the data that is going through.
- Since we are not concerned with the EHR data but mainly with the logs, we are assuming that the EHR data too will be encrypted using AES 256.
- Hence, confidentiality of all data is maintained and the privacy of the users is preserved.

Identification and authorization:

- Every user in the system needs to first prove their identity using their username and password. The password verification is done using a simple hash based check where the password entered is hashed and checked against a database of hashed passwords.
- If credentials do not match, the user cannot get into the web application, instead they are shown an invalid credential page.
- Once a user is identified, they are assigned one of the three roles (Patient, Doctor and Admin).
- Each role can only access the dashboard they are authorized to access.
- Each dashboard has the functionality only as needed for each user, hence demonstrating the concept of least privilege.

	Access EHR	Add EHR	Delete EHR	Access Logs
Patient	Own	No	No	Own
Doctor	All	Yes	Yes	No
Admin	No	No	No	All

- This table demonstrates the authorization by role:

Queries:

- The option to query is not made available to any user until they login.
- Only users that can prove their identity (successful authentication) are taken to their respective dashboards.
- Patients can query for their own audit data so they can see how their EHR data is being used.

- Admins (from the audit company) can query all patient audit data by name. This is because the audit company is managing the security of EHR data of the patient. If there is any unauthorized or wrong updates to the EHR data, they should be able to find it.
- Doctors are not authorized to query any audit data because they do not need to. They can however, query for the patients EHR data itself since they might need to make changes based on any of the patient's recent visits.

Immutability:

- Doctors can add or delete EHR data through the UI in the web application, but they are authorized to do so.
- No unauthorized changes can be made to the audit data. The admin can query audit data but still cannot make changes to it.
- If a person could make changes to the audit data, they could possibly access a patients EHR data and then delete the log stating that they did so.
- A master hash is calculated over the encrypted master audit file and stored in a database safely.
- If there are any additions to this file (authorized additions), that is, when any action is performed on the EHR data, the master hash is recalculated after the additions are made and replace the old master hash value.
- Every time the query button is hit, the current state of the encrypted master audit file is calculated and compared with the stored master hash.
- If it matches, the data is returned.
- If it does not match, the person querying is notified by the system.
- This ensures that any unauthorized changes are detected.
- Practically, when this system is being used, we can program the check to be ran every few minutes along with it being ran while querying.

Decentralization:

- Keeping only one copy of the master hash would create a single point of failure. If it gets compromised there is no way to know whether any of the data is legitimate or not.
- Hence, when this master hash is calculated or updated, the value is stored in the 2 databases on-prem (at different locations) and 3 databases in the cloud.
- When an integrity check is being performed, all the databases are checked.
- Inorder for the check to pass, 3 out of 5 of the database master hash values should match the current hash value calculated.
- We are leaving room for failure of 2 out of the 5 values due to cases where it might not match due to technical issues like syncing of all the stored master hash values.
- This ensures that a single compromised master hash value won't bring the availability of the system down.
- This also ensures that if an attacker is successful in compromising up to 2 of the values, the system would still perform integrity checks as intended.
- Having to compromise 3 different systems makes the attack impractical for the attacker and would require a lot of computational power and effort.

• Assumptions and Limitations

Assumptions:

- Our pieces of code for the authentication server, query server and audit server run on different machines and communicate with each.
- The EHR data is securely encrypted and the integrity is maintained.
- The hash of the passwords are stored in a database.
- The master hashes are stored in two on-premise databases and on three cloud databases. The hash values in these can be checked in an efficient time.
- The admin is from the audit company and their job is to investigate the audit files incase of any unauthorized access.
- In practice, the integrity check will be performed on the basis of a timer along with while querying, example, every 10 minutes. So the master hash will be checked against the hash that is calculated on the current audit file.
- We assume that the EHR data is stored as per the HIPAA regulation.
- The Audit Server and Query Server have the right public keys of each other. They get these values off a database in the organization that has the mapping of different components to their public keys.

Limitations:

- If the authentication server fails, there no user will be able to log in and hence will affect the availability of the system.
- There is no limit on the number of login attempts, hence, the system does not account for any brute force attacks, but it is a functionality that can be easily added.
- If a person, specially one with higher privilege like doctor or admin gets their password compromised, there is no second factor of authentication. This too can be easily added and can use an application like DUO.
- As the system is right now, the integrity check is performed only when the audit data is queried. This could leave a long time between when we detect any unauthorized changes. However, when we implement the time based check, this limitation will be overcome.

- **Implementation**

Login.py:

This is the program that runs first. This program handles authentication and hence, can be considered as the “authentication server”. It includes the following components:

- **Import hashlib**
We are using the functions in the hashlib library to calculate the SHA-256 hash.
- **Import flask**
We are using the functions from the flask library to handle the web application.
- **hash_password(password):**
Purpose: Converts a plaintext password into a SHA-256 hash for secure storage and comparison.
Parameters: password - The plaintext password to be hashed.
Returns: Hashed password using SHA-256.
- **is_authenticated(request):**
Purpose: Checks if the current user session is authenticated by verifying if their username is stored in a session cookie and exists in the users dictionary.
Parameters: request - The Flask request object containing cookies.
Returns: Boolean indicating whether the user is authenticated.

Routes and Views

- **home():**
Endpoint: '/' (GET)
Purpose: Renders the login page.
Returns: The HTML template for the login page.
- **login():**
Endpoint: '/login' (POST)
Purpose: Processes login requests. It checks submitted credentials against stored hashes, sets a session cookie if the credentials are valid, and redirects to the appropriate dashboard based on user role.
Returns: Redirection to the relevant dashboard on successful login or a 403 error with "Invalid credentials" if login fails.
- **patientDashboard():**
Endpoint: '/patientDashboard' (GET)
Purpose: Displays the patient dashboard if the user is authenticated as a patient.
Returns: The patient dashboard HTML page or redirects to the login page if authentication fails.
- **adminDashboard():**
Endpoint: '/adminDashboard' (GET)
Purpose: Displays the admin dashboard if the user is authenticated as an admin.

Returns: The admin dashboard HTML page or redirects to the login page if authentication fails.

- **doctorDashboard():**

Endpoint: '/doctorDashboard' (GET)

Purpose: Displays the doctor dashboard if the user is authenticated as a doctor.

Returns: The doctor dashboard HTML page or redirects to the login page if authentication fails.

- **adminQuery():**

Endpoint: '/adminQuery' (POST)

Purpose: Allows admins to query patient information and their audit logs.

Returns: Updates the admin dashboard HTML page with queried patient information and audit logs.

- **doctorQuery():**

Endpoint: '/doctorQuery' (POST)

Purpose: Allows doctors to query information for a specific patient.

Returns: Updates the doctor dashboard HTML page with the queried patient information.

- **addComment():**

Endpoint: '/addComment' (POST)

Purpose: Allows authenticated users to add a health record comment for a specific patient.

Returns: Updates the admin dashboard HTML page with the status of the comment addition.

- **deleteComment():**

Endpoint: '/deleteComment' (POST)

Purpose: Allows authenticated users to delete a specific comment from a patient's health record.

Returns: Updates the admin dashboard HTML page with the status of the comment deletion.

- **patientQuery():**

Endpoint: '/patientQuery' (POST)

Purpose: Allows patients to view their own medical records and audit logs.

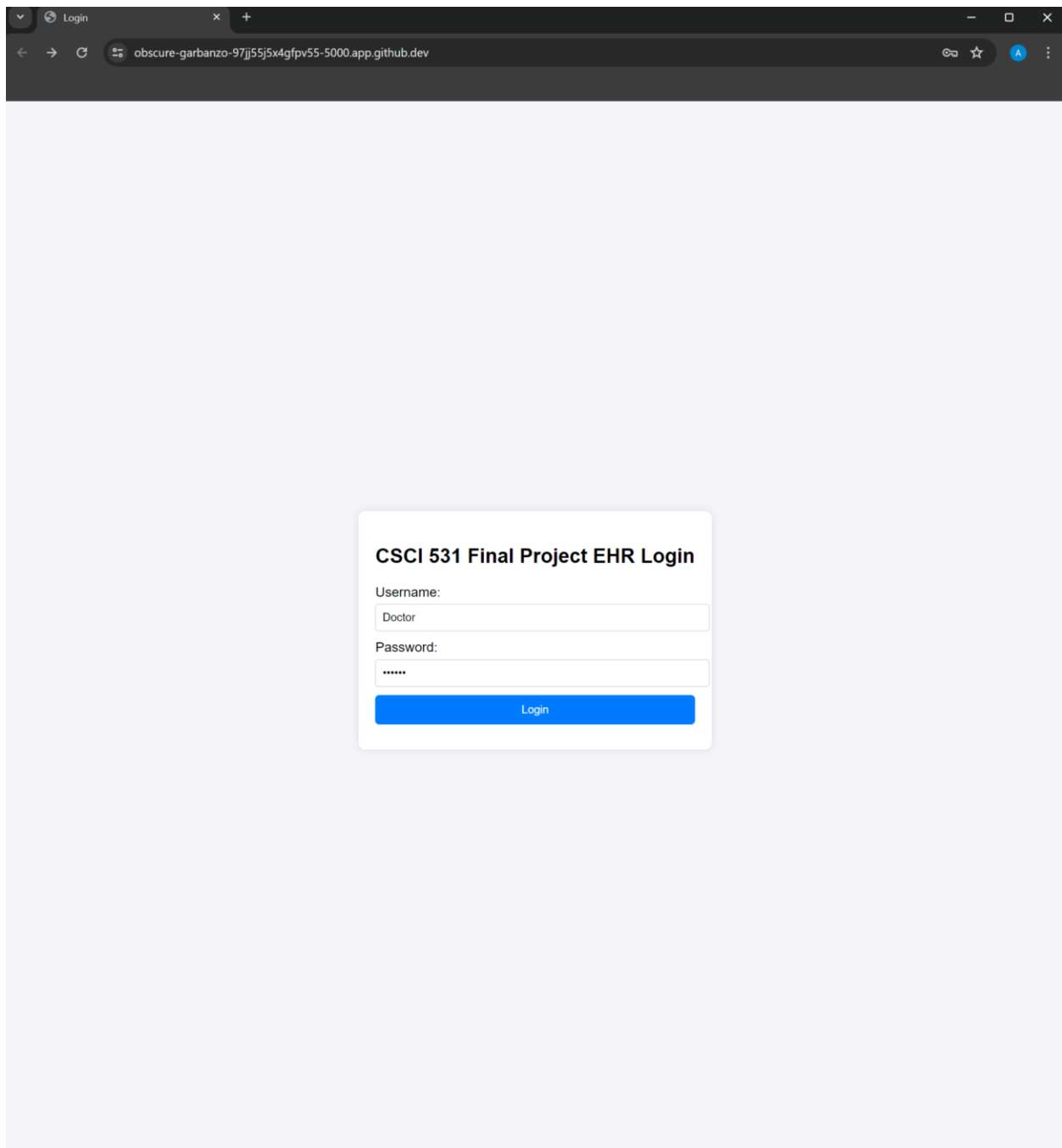
Returns: Updates the patient dashboard HTML page with the patient's own information and audit logs.

- **logout():**

Endpoint: '/logout' (GET)

Purpose: Logs out the user by deleting the 'username' cookie and redirects to the home page.

Returns: Redirect to the home page after clearing the session cookie.



Screenshot of successful login

Dashboard

obs-cure-garbanzo-97jj55j5x4gfpv55-5000.app.github.dev/doctorDashboard

Doctor Dashboard

Enter Patient's Name:

Search

No results found or no query made.

Add Comment

Patient's Name for Comment:

Add Comment to EHR:

Write your comment here...

Add Comment

Delete Comment

Patient's Name for Comment Deletion:

Comment to Delete:

Enter Comment

Delete Comment

Screenshot of successful login



Screenshot of unsuccessful login

Query.py:

This program that is used to take queries that the user enters on their dashboard after they have been authenticated. This program will run on our query server. This includes querying EHR records, querying audit logs, adding comments to EHR data, deleting comments from EHR data based on their role. This program communicates with Audit.py (audit server).

- **Import crypt**

From assignment 3, we are importing the functions to perform RSA+AES encryptions and decryptions.

- **Import**

Imports functions from the audit server program.

audit

- **auditServerPublicKey:**

This is used to store the location of the public key of the audit server to encrypt communications from the query server to the audit server.

- **queryServerPrivateKey:**

This is used to store the location of the private key of the audit server to decrypt communications from the audit server to the query server.

- **queryPatient(name, user):**

Purpose: Fetches the EHR data for a given patient and logs the query to the audit file.

Parameters:

- name: The name of the patient whose records are being queried.
- user: The identifier of the user making the query.

Process:

- Encrypts the patient's name and the user's identifier using the public key of the audit server.
- Reads the EHR data from a file.
- Logs the access by calling addRecord with the encrypted data.

Returns: The contents of the EHR file as a string.

- **checkPatientAudit(name):**

Purpose: Retrieves and decrypts the audit logs for a given patient.

Parameters:

- name: The name of the patient whose audit logs are being checked.

Process:

- Encrypts the patient's name using the public key.
- Checks the audit logs using checkAudit, passing the encrypted name.
- Decrypts the returned audit logs using the private key of the query server.

Returns: The decrypted audit log data.

- **addHealthRecord(name, user, comment):**

Purpose: Adds a new health record or comment to a patient's file and logs the action.

Parameters:

- name: The name of the patient.
- user: The identifier of the user making the addition.
- comment: The health record or comment to be added.

Process:

- Encrypts the patient name, user identifier, and the comment.
- Appends the comment to the patient's EHR file.
- Logs the addition to the audit file using addedComment.

Returns: Confirmation message stating that the comment has been added.

- **deleteHealthRecord(name, user, comment):**

Purpose: Deletes a specific comment or record from a patient's EHR file and logs the action.

Parameters:

- name: The name of the patient.
- user: The identifier of the user performing the deletion.
- comment: The specific comment to be removed.

Process:

- Encrypts the patient name, user identifier, and the comment.
- Reads the patient's EHR file, removes the line containing the comment, and writes the updated data back to the file.
- Logs the deletion to the audit trail using deletedComment.

Returns: Confirmation message stating that the comment has been deleted.

Dashboard

obscore-garbanzo-97/jj55j5x4gfpv55-5000.app.github.dev/doctorQuery

Doctor Dashboard

Enter Patient's Name:

Search

Results for "John Doe":

Born January 1 1990
is sick
Has an illness

Add Comment

Patient's Name for Comment:

Add Comment to EHR:

Write your comment here...

Add Comment

Delete Comment

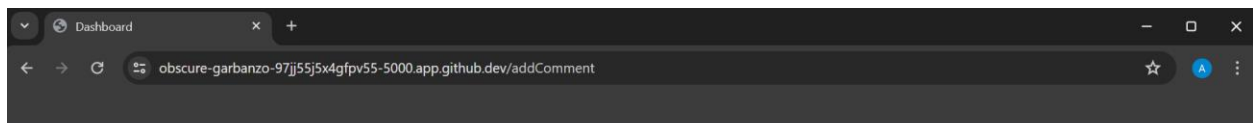
Patient's Name for Comment Deletion:

Comment to Delete:

Enter Comment

Delete Comment

Screenshot of doctor querying EHR



Doctor Dashboard

Enter Patient's Name:

Search

No results found or no query made.

Add Comment

Patient's Name for Comment:

Add Comment to EHR:

Add Comment

is overweight has been added.

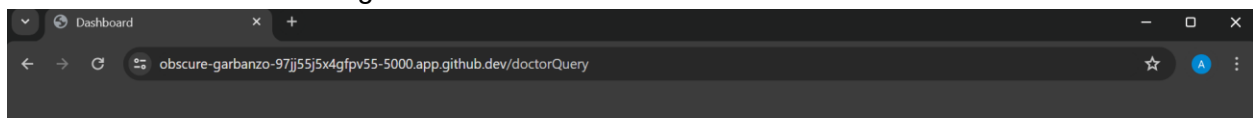
Delete Comment

Patient's Name for Comment Deletion:

Comment to Delete:

Delete Comment

Screenshot of doctor adding to EHR



Doctor Dashboard

Enter Patient's Name:

Search

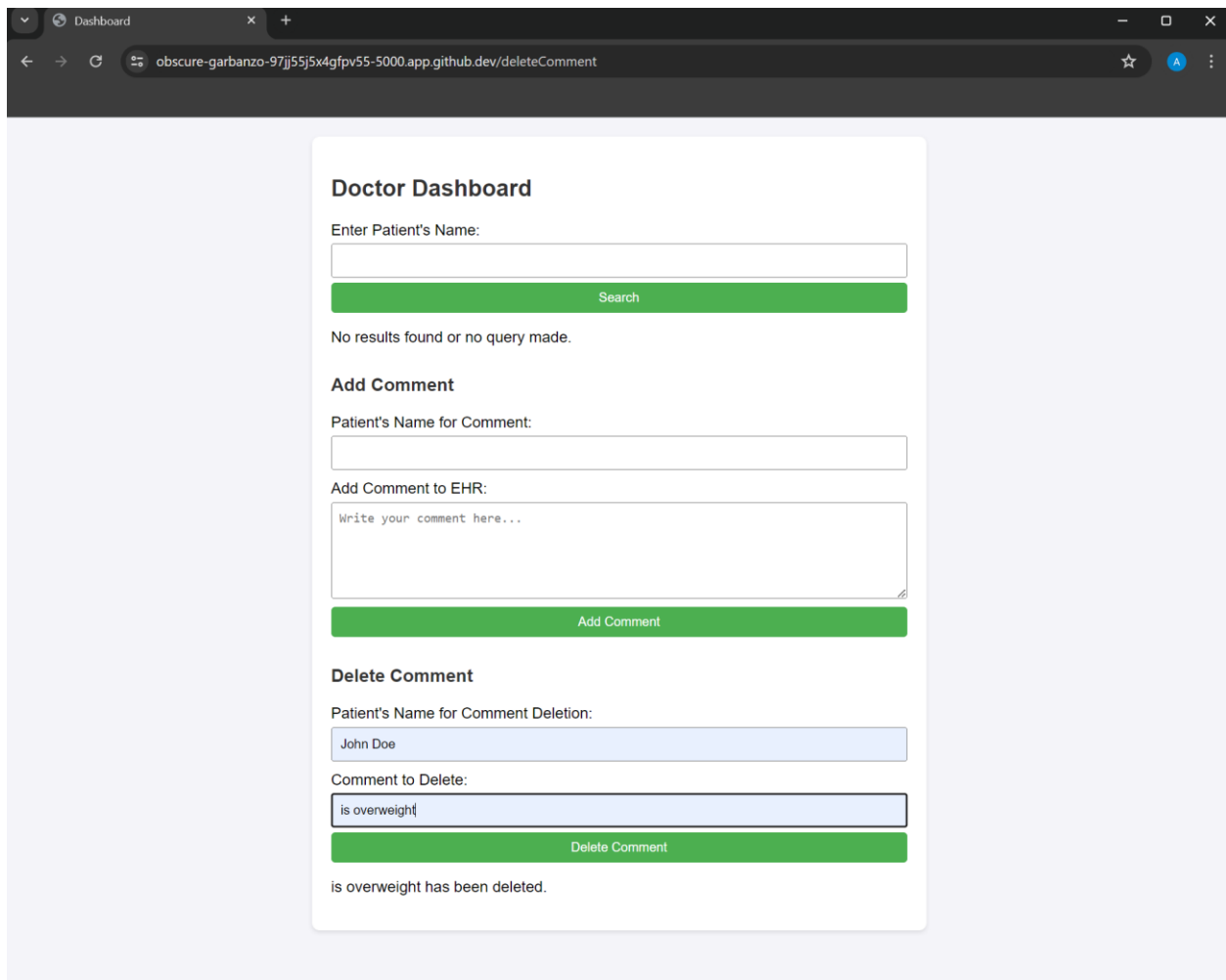
Results for "John Doe":

Born January 1 1990
is sick
Has an illness
is overweight

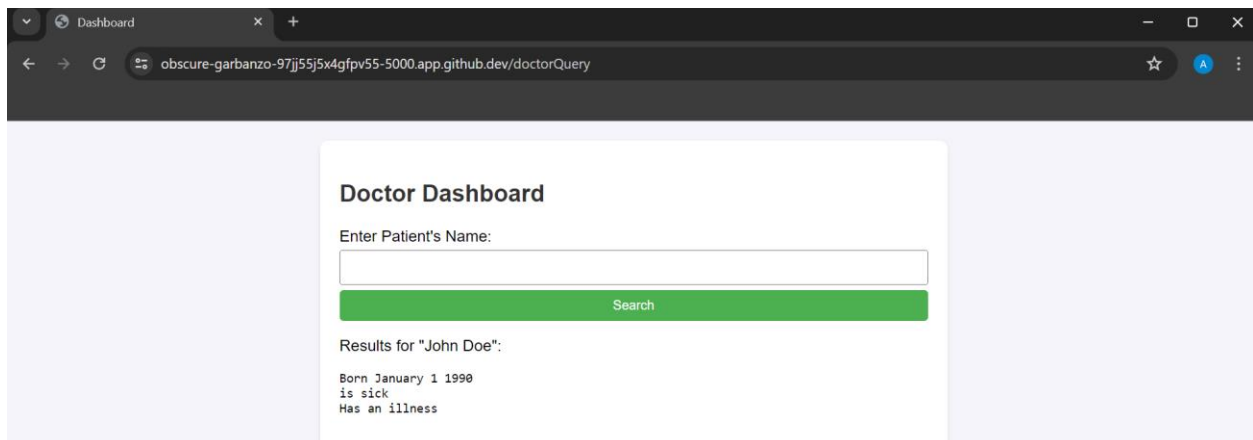
Add Comment

Patient's Name for Comment:

Screenshot of added EHR



Screenshot of doctor removing EHR



Screenshot of removed EHR

Audit.py

This program is used to handle all access and updates made to the audit file. This is the program that will run on our audit server. It receives communications from query.py (query server).

- **Import datetime**
We are using functions from the datetime library to note down the time while entering the logs.
- **Import crypt**
From assignment 3, we are importing the functions to perform RSA+AES encryptions and decryptions.
- **Import hashlib**
We are using the functions in the hashlib library to calculate the SHA-256 hash.
- **auditPrivateKeyFile**
Location of the private key of the audit server, used to decrypt communications from the query server.
- **auditPublicKeyFile**
Location of the public key of the audit server.
- **queryPublicKeyFile**
Location of the public key of the query server, used to encrypt communications from the audit server to the query server.
- **encryptedAuditRecord**
Location of the stored audit file after AES encryption.
- **decryptedAuditRecord**
Location at which the decrypt audit file will be stored.
- **updateHash(auditFile):**
Purpose: Updates the cryptographic hash of the audit record for integrity checks.
Parameters:
 - auditFile: The path to the audit file whose hash needs to be updated.**Process:**
 - Computes the SHA-1 hash of the file's contents and encrypts this hash using the public key of the audit server.
- **checkHash(auditFile):**
Purpose: Checks the integrity of the audit record by comparing the current hash of the file against a previously stored hash.
Parameters:
 - auditFile: The path to the audit file to check.

Process:

- Decrypts the stored hash using the private key, computes the current hash of the file, and compares the two to detect any tampering.

- **encryptAuditRecord(auditFile):**

Purpose: Encrypts the audit record file.

Parameters:

- auditFile: The path to the audit file that needs to be encrypted.

Returns: The path to the encrypted audit record file.

- **decryptAuditRecord(auditFile):**

Purpose: Decrypts the encrypted audit record file.

Parameters:

- auditFile: The path to the encrypted audit file that needs to be decrypted.

Process:

Decrypts the file and writes the contents to decryptedAuditRecord.txt.

- **addRecord(name, user):**

Purpose: Logs an access event to the audit record.

Parameters:

- name: The encrypted name of the patient whose record was accessed.
- user: The encrypted username of the individual accessing the record.

Process:

- Decrypts the audit record, checks its integrity, logs the access event with a timestamp, updates the hash, and re-encrypts the audit record.

- **addedComment(name, user, comment):**

Purpose: Logs a comment addition event to the audit record.

Parameters:

- name: The encrypted name of the patient to whom the comment was added.
- user: The encrypted username of the user who added the comment.
- comment: The encrypted comment that was added.

Process:

- Similar to addRecord, but logs a comment addition event.

- **deletedComment(name, user, comment):**

Purpose: Logs a comment deletion event to the audit record.

Parameters:

- name: The encrypted name of the patient from whose record the comment was deleted.
- user: The encrypted username of the user who deleted the comment.
- comment: The encrypted comment that was deleted.

Process:

- Similar to addRecord, but logs a comment deletion event.

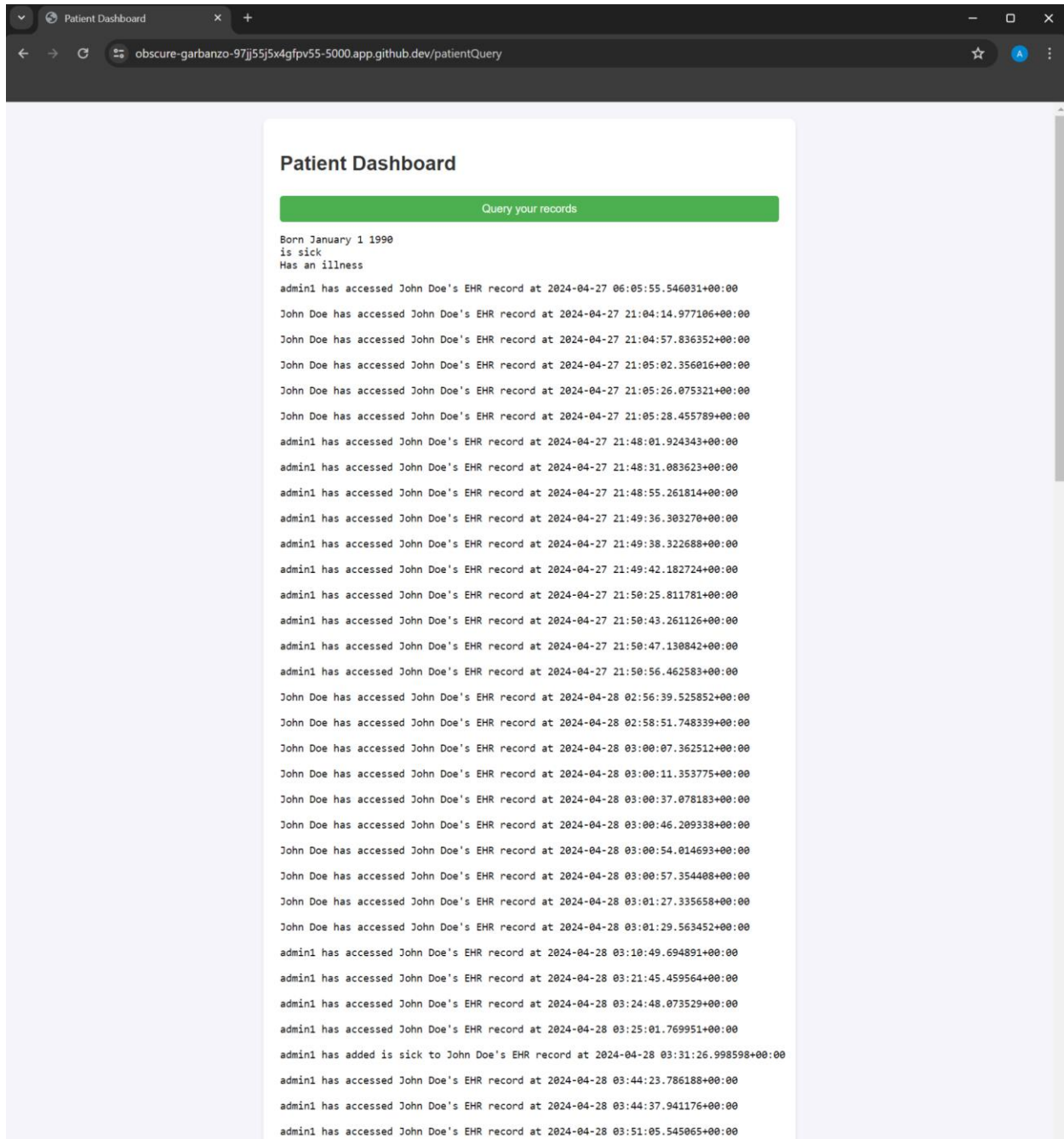
- **checkAudit(name):**

Purpose: Retrieves and filters audit records related to a specific patient.

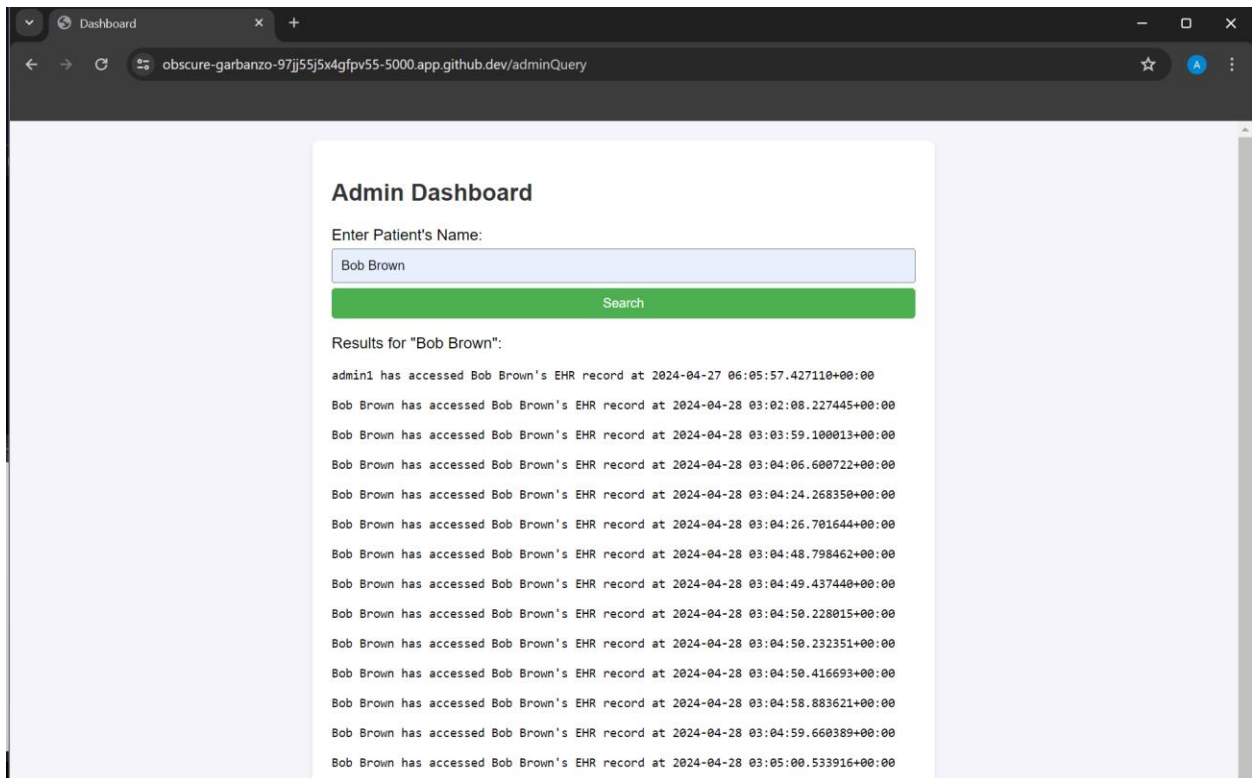
Parameters:

- name: The encrypted name of the patient whose audit records are to be checked.

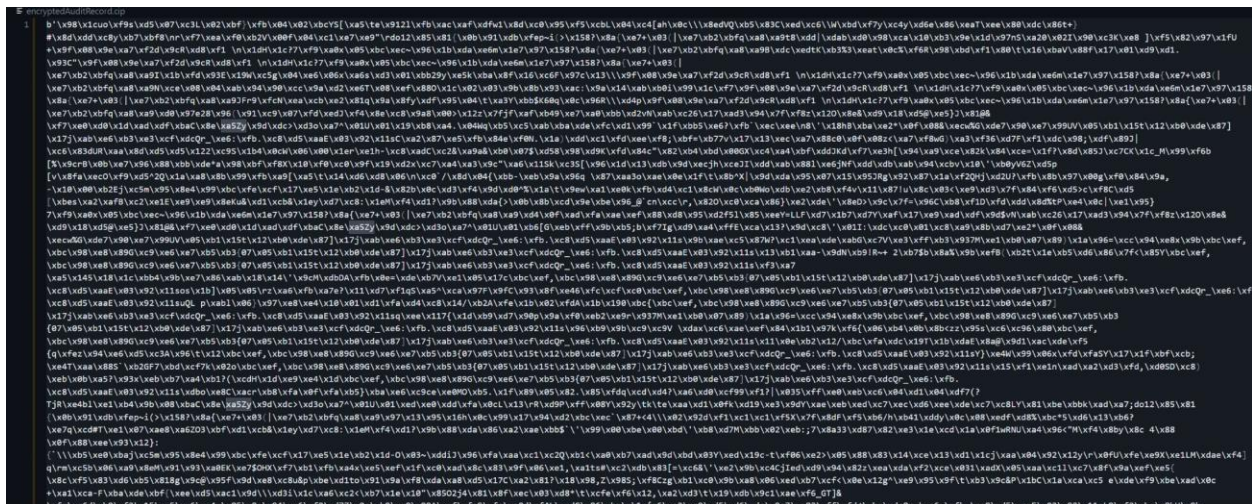
Returns: Encrypted audit records relevant to the specified patient.



Screenshot of patient checking their own audit record



Screenshot of an authorized admin being able to query the audit record



Screenshot of encrypted audit record



Screenshot of encrypted hash of the audit record

● References

- Class Presentation on Homographic Encryption (Joseph Couri and Malhar Shenolkar)
https://piazza.com/class_profile/get_resource/lqyim3vl7sg320/lut11wwg4re2wv
- PQC: Kyber
<https://en.wikipedia.org/wiki/Kyber>
- A. Pathak, T. Patil, S. Pawar, P. Raut and S. Khairnar, "Secure Authentication using Zero Knowledge Proof," *2021 Asian Conference on Innovation in Technology (ASIANCON)*, PUNE, India, 2021, pp. 1-8, doi: 10.1109/ASIANCON51346.2021.9544807.
- Project Document:
https://piazza.com/class_profile/get_resource/lqyim3vl7sg320/lqznn9ofdf83sw